Curso Autônomo de ADS: Plano de Estudos

Oct 26, 2026 · @Alexandre

Visão geral

Este plano complementa a faculdade de Ciência da Computação com 5 semestres de estudo prático: cerca de 580 horas em aproximadamente 2 anos e meio, para ganhar prática e chegar preparado ao mercado.

- **Ritmo:** 1 hora por dia, de segunda a sexta, uma matéria por dia.
- **Aula:** 2 sessões de 1 hora, uma de teoria e uma de prática.
- **Duração:** o Semestre 1 tem 10 aulas por matéria (cerca de 20 semanas); do Semestre 2 ao 5, 12 aulas (cerca de 24 semanas).
- **Ambiente:** Windows; do Semestre 2 em diante, WSL2 com Ubuntu para Sistemas Operacionais, Redes e Docker.
- **Diploma:** o plano não substitui a graduação; ele reforça a faculdade.
- **Revisão:** ferramentas e versões mudam, então vale revisar a lista de cada semestre antes de começá-lo.

Semestre	Foco	Aulas por matéria
1	Fundações: C, Matemática Discreta, Arquitetura, Java e Banco de Dados	10
2	Sistemas Operacionais, Algoritmos, Redes, Engenharia de Software e Back-end	12
3	Front-end, Banco de Dados avançado, Python e Estatística, Segurança e Cloud	12
4	IA e ML, Compiladores, Arquitetura de Software, Mobile e Engenharia de Dados	12
5	Projeto final, entrevistas, Go e carreira	12

Semestre 1: Fundações

Cinco matérias, uma por dia, com 10 aulas cada. Termina com um projeto final por matéria (biblioteca de estruturas em C, sistema em Java e banco de dados real).

Segunda: Estruturas de Dados em C

1. Revisão de C: tipos, funções, escopo, compilação com gcc e depuração básica
2. Ponteiros I: endereços, `&` e `*`, passagem por referência
3. Vetores, strings, matrizes, `struct` e aritmética de ponteiros
4. Memória dinâmica (heap): `malloc`, `calloc`, `realloc`, `free`, vazamentos e Valgrind
5. Listas ligadas simples e duplas
6. Pilhas e filas, com vetor e com lista, e suas aplicações
7. Recursão e análise de complexidade (Big-O)
8. Árvores binárias e árvores binárias de busca
9. Ordenação e busca: insertion, merge, quick e busca binária
10. Tabelas hash e projeto final (uma mini biblioteca de estruturas de dados)

Terça: Matemática Discreta

1. Lógica proposicional: conectivos, tabelas-verdade e equivalências
2. Álgebra booleana e portas lógicas
3. Lógica de predicados e quantificadores
4. Técnicas de demonstração: direta, contrapositiva e por absurdo
5. Indução matemática e definições recursivas
6. Conjuntos, relações e funções
7. Combinatória: contagem, permutações e combinações
8. Aritmética modular e relações de recorrência
9. Grafos I: conceitos, representações, BFS e DFS
10. Grafos II: caminhos mínimos (Dijkstra), árvores geradoras e revisão geral

Quarta: Arquitetura de Computadores

1. Bits e bytes, bases numéricas, inteiros e ponto flutuante
2. Lógica digital: circuitos combinacionais e sequenciais
3. Modelo de Von Neumann: CPU, barramentos e ciclo busca-decodifica-executa
4. Conjunto de instruções (ISA): RISC vs CISC e assembly básico
5. Assembly na prática: registradores, desvios, pilha de chamadas e o código C visto como assembly
6. Hierarquia de memória: registradores, cache, RAM, disco e localidade
7. Cache em detalhe: mapeamento, hit/miss e impacto no desempenho
8. Pipeline e paralelismo: hazards e multicore
9. Entrada e saída: interrupções, DMA e periféricos

10. Desempenho (CPI, Lei de Amdahl) e visão integrada, do código ao hardware

Quinta: Programação Orientada a Objetos em Java

1. Fundamentos de Java: JDK, JVM, sintaxe, tipos, controle de fluxo e métodos
2. Classes e objetos: atributos, métodos, construtores e `this`
3. Encapsulamento: modificadores de acesso e imutabilidade
4. Herança, `super` e sobrescrita
5. Polimorfismo e classes abstratas
6. Interfaces e composição vs herança
7. Coleções e Generics: `ArrayList`, `HashMap` e `Set`
8. Exceções, leitura e escrita de arquivos, e testes com JUnit
9. UML de classes e princípios SOLID
10. Projeto final (por exemplo, um gerenciador de biblioteca), publicado no GitHub com Git

Sexta: Sistemas de Informação e Banco de Dados

1. Sistemas de informação e processos de negócio, e o papel dos dados
2. Conceitos de banco de dados: SGBD, modelo relacional, chaves e instalação do PostgreSQL
3. Modelagem conceitual: MER, entidades, relacionamentos e cardinalidades
4. Do MER ao modelo relacional
5. SQL básico: DDL e DML, `SELECT`, `WHERE`, `ORDER BY` e agregações
6. SQL intermediário: `JOIN`, subconsultas, `GROUP BY` e `HAVING`
7. Normalização (1FN a 3FN) e dependências funcionais
8. Índices, views, transações e ACID
9. Java com banco de dados via JDBC
10. Projeto final: modelar, criar e consultar o banco de um sistema real

Semestre 2: Sistemas, Redes e Back-end

Cinco matérias com 12 aulas cada. A quinta e a sexta terminam no mesmo projeto: planejado na quinta, construído na sexta. Requer Linux (WSL2 no Windows) para Sistemas Operacionais e Docker.

Segunda: Sistemas Operacionais

1. O que é um SO: kernel, modo usuário e modo kernel e chamadas de sistema
2. Linux na prática: shell, pipes, redirecionamento e scripts bash

3. Processos: PCB, estados, `fork` e `exec` em C
4. Threads e concorrência com `pthreads`
5. Escalonamento de CPU: FCFS, SJF, Round Robin e prioridades
6. Sincronização I: condições de corrida, seção crítica, mutex e semáforos
7. Sincronização II: deadlock, produtor-consumidor e jantar dos filósofos
8. Gerência de memória: endereçamento, paginação e segmentação
9. Memória virtual: page fault, substituição de páginas (LRU) e TLB
10. Sistemas de arquivos: inodes, diretórios, journaling e permissões
11. Entrada e saída, drivers e comunicação entre processos (pipes, sinais e memória compartilhada)
12. Projeto final: um mini shell em C

Terça: Algoritmos e Estruturas de Dados Avançados (em Java)

1. Análise de algoritmos: Big-O, Theta, Omega e recorrências (Teorema Mestre)
2. Divisão e conquista
3. Árvores balanceadas: AVL e rotações
4. Árvores B e Trie (usadas em bancos de dados e autocompletar)
5. Heaps, filas de prioridade e heapsort
6. Algoritmos gulosos: seleção de intervalos e códigos de Huffman
7. Programação dinâmica I: Fibonacci, troco e mochila
8. Programação dinâmica II: maior subsequência comum e distância de edição
9. Grafos avançados: Dijkstra com heap, Bellman-Ford, Floyd-Warshall, Kruskal, Prim e union-find
10. Backtracking (N-rainhas, sudoku) e busca em strings (KMP)
11. Complexidade: P, NP, NP-completo, heurísticas e algoritmos de aproximação
12. Projeto: resolução guiada de problemas no LeetCode ou Beecrowd e um mini projeto de menor caminho em mapa

Quarta: Redes de Computadores

1. Visão geral: comutação de pacotes, modelos OSI e TCP/IP, latência e vazão
2. Camada de aplicação: HTTP, HTTPS, DNS e e-mail, com `curl` e o navegador
3. Programação com sockets: cliente e servidor TCP e UDP
4. Camada de transporte: TCP vs UDP, controle de fluxo e de congestionamento
5. Camada de rede I: IPv4, sub-redes e CIDR
6. Camada de rede II: NAT, DHCP, ICMP e IPv6

7. Roteamento: RIP, OSPF e BGP, e a relação com os grafos da terça
8. Camada de enlace: Ethernet, ARP, switches e Wi-Fi
9. Análise de tráfego com Wireshark, `ping`, `traceroute` e `dig`
10. Segurança: TLS, certificados, firewalls e noções de criptografia
11. Redes na prática moderna: proxy reverso, balanceadores de carga, CDN e VPN
12. Projeto final: chat cliente-servidor com sockets, analisado no Wireshark

Quinta: Engenharia de Software

1. Ciclo de vida e processos: cascata, ágil, Scrum e Kanban
2. Requisitos: funcionais e não funcionais, histórias de usuário e casos de uso
3. Modelagem com UML: casos de uso, sequência e componentes
4. Git e GitHub avançados: branches, pull requests e code review
5. Padrões de projeto I: criacionais (Singleton, Factory e Builder)
6. Padrões de projeto II: estruturais e comportamentais (Strategy, Observer e Decorator)
7. Testes I: unitários, TDD e mocks
8. Testes II: integração, testes de ponta a ponta e cobertura
9. Arquitetura: camadas, MVC e monolito vs microsserviços
10. Qualidade e manutenção: refatoração, code smells e Clean Code
11. CI/CD com GitHub Actions, versionamento e documentação
12. Projeto integrador: backlog, modelagem e testes do sistema que será construído na sexta

Sexta: Back-end e Sistemas Distribuídos

1. HTTP e REST: verbos, status codes e JSON, testando com `curl` e Postman
2. Spring Boot: primeiro projeto e primeiros endpoints
3. Persistência com Spring Data JPA e PostgreSQL
4. API completa: validação, tratamento de erros, DTOs e documentação com OpenAPI
5. Autenticação e autorização: hash de senhas e JWT
6. Fundamentos de sistemas distribuídos: falhas, latência, consistência e teorema CAP
7. Comunicação entre serviços: RPC e mensageria (RabbitMQ e Kafka)
8. Replicação e particionamento (sharding)
9. Cache e desempenho com Redis
10. Contêineres: Docker e Docker Compose, subindo API e banco juntos
11. Observabilidade e deploy: logs, métricas e publicação em um serviço de nuvem
12. Projeto integrador: implementar a API planejada na quinta, com testes, Docker e CI

Semestre 3: Full stack, dados, segurança e nuvem

O front-end da segunda consome a API do Semestre 2, a segurança da quinta audita esse mesmo sistema e a nuvem da sexta o publica. Cloud e DevOps é a matéria mais pesada; se ficar insustentável, pode ir para o Semestre 4.

Segunda: Desenvolvimento Front-end

1. Como a web funciona no navegador: HTML semântico, acessibilidade e DevTools
2. CSS I: seletores, box model e Flexbox
3. CSS II: Grid, responsividade e mobile-first
4. JavaScript I: sintaxe, tipos, funções, escopo, arrays e objetos
5. JavaScript II: DOM, eventos, Promises, `async/await` e `fetch`
6. JavaScript III: módulos, recursos do ES6+, npm e Vite
7. TypeScript: tipos, interfaces e generics
8. React I: componentes, JSX, props e estado (`useState`)
9. React II: hooks (`useEffect`), formulários e roteamento
10. Consumindo a API do Semestre 2: login com JWT, gerenciamento de estado e React Query
11. Testes e qualidade: Vitest, Testing Library, desempenho e acessibilidade (Lighthouse)
12. Projeto: interface completa para a API construída no Semestre 2

Terça: Banco de Dados Avançado

1. SQL avançado: CTEs e window functions
2. Funções, stored procedures e triggers no PostgreSQL
3. Índices a fundo: B-tree, índices compostos e `EXPLAIN ANALYZE`
4. Otimização de consultas: planos de execução e anti-padrões (como o N+1)
5. Transações e concorrência: níveis de isolamento, locks, MVCC e deadlocks
6. Por dentro do SGBD: páginas, WAL e recuperação de falhas
7. NoSQL I: chave-valor e documentos (MongoDB)
8. NoSQL II: colunar, grafos e séries temporais, e quando usar cada um
9. Replicação, particionamento, backup e restauração
10. Modelagem dimensional: data warehouse, ETL e OLTP vs OLAP
11. Migrations e versionamento de esquema (Flyway) e acesso a dados em produção
12. Projeto: um banco com grande volume de dados, com benchmark antes e depois da otimização

Quarta: Python, Probabilidade e Estatística Aplicada

1. Python I: sintaxe, estruturas de dados, funções e ambientes virtuais
2. Python II: orientação a objetos, módulos, arquivos, exceções e testes com `pytest`
3. NumPy e vetorização
4. Pandas I: DataFrames, leitura e limpeza de dados
5. Pandas II: agrupamentos, junções e séries temporais
6. Visualização de dados: Matplotlib e Seaborn
7. Probabilidade: probabilidade condicional e teorema de Bayes
8. Variáveis aleatórias e distribuições: binomial, Poisson e normal
9. Estatística descritiva e inferência: amostragem, intervalos de confiança e Teorema Central do Limite
10. Testes de hipótese, correlação e regressão linear
11. Coleta de dados: APIs, web scraping, SQL com Python e automação
12. Projeto: análise exploratória de um conjunto de dados real, entregue como notebook com relatório

Quinta: Segurança da Informação

1. Fundamentos: confidencialidade, integridade e disponibilidade, riscos e modelagem de ameaças
2. Criptografia I: simétrica, hash e MAC
3. Criptografia II: chave pública, RSA, assinaturas digitais e PKI
4. Autenticação e identidade: senhas, MFA, OAuth2 e OpenID Connect
5. Segurança web I: OWASP Top 10, injeção de SQL, XSS e CSRF
6. Segurança web II: controle de acesso, sessões, cabeçalhos de segurança e CORS
7. Segurança de sistemas: hardening de Linux, privilégios e conceitos de malware
8. Segurança de redes: firewalls, IDS/IPS, VPN e tipos de ataque (estudados em laboratório)
9. Testes de segurança: análise estática e dinâmica, dependências vulneráveis e prática em laboratórios legais (DVWA, Juice Shop e TryHackMe)
10. Desenvolvimento seguro: gestão de segredos e análise de segurança no CI
11. Privacidade e legislação: LGPD
12. Projeto: auditar e corrigir vulnerabilidades da API e do front-end, com relatório

Sexta: Cloud e DevOps

1. Computação em nuvem: IaaS, PaaS e SaaS, modelos de custo e segurança da conta

2. Serviços essenciais da AWS: computação (EC2), armazenamento (S3) e redes (VPC)
3. Serviços gerenciados: banco (RDS), filas e funções serverless (Lambda)
4. Linux para operações: scripts de shell, SSH e `systemd`
5. Infraestrutura como código com Terraform
6. CI/CD avançado: ambientes e estratégias de deploy (blue-green e canary)
7. Kubernetes I: pods, deployments e services
8. Kubernetes II: configuração, volumes, Helm e escalonamento
9. Observabilidade: Prometheus, Grafana, logs centralizados e tracing
10. Confiabilidade: SLI e SLO, resposta a incidentes, backups e controle de custos
11. Segurança na nuvem: IAM, gestão de segredos e menor privilégio
12. Projeto: publicar na nuvem o full stack (front-end e API) com infraestrutura como código e pipeline

Semestre 4: IA, teoria e especialização

O semestre mais pesado em teoria. A API dos semestres anteriores ganha um app mobile, passa a ser tratada como sistema distribuído de verdade e pode servir um modelo de IA, e os dados que ela gera alimentam o pipeline da sexta. A Engenharia de Dados é a matéria mais fácil de mover de semestre.

Segunda: Inteligência Artificial e Machine Learning

1. Panorama: IA, ML e deep learning, tipos de aprendizado, o fluxo de um projeto de ML e o scikit-learn
2. Matemática essencial com NumPy: vetores, matrizes, derivadas e gradiente
3. Regressão linear e gradiente descendente feitos do zero
4. Classificação: regressão logística, k-NN e métricas (precisão, recall, F1 e ROC)
5. Árvores de decisão, Random Forest e boosting (XGBoost)
6. Generalização: overfitting, validação cruzada, regularização e viés-variância
7. Engenharia de atributos, pipelines e dados desbalanceados
8. Aprendizado não supervisionado: k-means e PCA
9. Redes neurais I: perceptron e backpropagation do zero
10. Redes neurais II: PyTorch e redes convolucionais para imagens
11. NLP e LLMs: embeddings, transformers, uso de APIs, RAG, ética e viés
12. Projeto: um problema real de ponta a ponta, dos dados ao modelo servido por uma API

Terça: Linguagens Formais, Compiladores e Teoria da Computação

1. Alfabetos, linguagens, gramáticas e hierarquia de Chomsky
2. Autômatos finitos e expressões regulares (AFD e AFN)
3. Análise léxica: construindo um lexer
4. Gramáticas livres de contexto e autômatos de pilha
5. Análise sintática descendente: LL e descida recursiva
6. Análise sintática ascendente: LR e geradores de analisadores (ANTLR)
7. Árvore sintática abstrata (AST), análise semântica, tabela de símbolos e tipos
8. Representações intermediárias e geração de código
9. Otimização de código e alocação de registradores
10. Máquinas virtuais e interpretadores: bytecode, JVM e coleta de lixo
11. Máquinas de Turing, decidibilidade e o problema da parada
12. Projeto: criar uma linguagem de programação simples com seu próprio interpretador

Quarta: Arquitetura de Software e Sistemas Distribuídos Avançados

1. Atributos de qualidade, trade-offs, registros de decisão arquitetural (ADR) e o modelo C4
2. Domain-Driven Design I: linguagem ubíqua e contextos delimitados
3. Domain-Driven Design II: agregados, value objects, arquitetura hexagonal e arquitetura limpa
4. Microsserviços: quando e como dividir, comunicação e API gateway
5. Arquitetura orientada a eventos: Kafka, event sourcing e CQRS
6. Consistência distribuída: padrão Saga, idempotência e outbox
7. Coordenação e consenso: Raft e Paxos (conceitos), relógios lógicos e eleição de líder
8. Resiliência: circuit breaker, retry, timeouts e chaos engineering
9. Escalabilidade e desempenho: balanceamento, filas e teste de carga com k6
10. Design de sistemas I: estudos de caso (encurtador de URL e feed de notícias)
11. Design de sistemas II: chat, streaming e estimativa de capacidade
12. Projeto: evoluir o sistema do Semestre 3 para uma arquitetura orientada a eventos com 2 ou 3 serviços

Quinta: Desenvolvimento Mobile (React Native e TypeScript)

1. Ecossistema mobile: nativo vs multiplataforma, React Native, Expo e o ambiente no Windows com emulador Android
2. Fundamentos: componentes, estilização e Flexbox no mobile

3. Navegação: stack, tabs e drawer com Expo Router
4. Estado e formulários: hooks, Context e validação
5. Consumo de API, autenticação e armazenamento seguro de tokens
6. Armazenamento local: SQLite e abordagem offline-first
7. Recursos nativos: câmera, localização, notificações push e permissões
8. UI/UX mobile: acessibilidade, animações e diretrizes de design
9. Desempenho e depuração: listas grandes, memória e profiling
10. Testes mobile: unitários, de componentes e ponta a ponta (Maestro)
11. Build e publicação: EAS Build, assinatura e distribuição de testes
12. Projeto: um app completo que consome a API dos semestres anteriores

Sexta: Engenharia de Dados

1. Panorama: ciclo de vida dos dados e arquitetura de dados moderna
2. Pipelines de ETL e ELT em Python
3. Data lake e lakehouse: formatos Parquet e Delta, e camadas bronze, silver e gold
4. SQL analítico com DuckDB
5. Orquestração com Apache Airflow
6. Processamento em lote com Spark e PySpark I
7. Spark II: otimização e particionamento
8. Streaming: Kafka e processamento em tempo real
9. Qualidade e governança de dados: testes de dados e Great Expectations
10. Transformação com dbt e modelagem analítica
11. Dados na nuvem e BI: dashboards com Metabase ou Looker Studio, e custos
12. Projeto: pipeline completo, da ingestão de uma API ou stream até o dashboard

Semestre 5: Projeto final e carreira

A segunda e a sexta formam o projeto final: da ideia ao MVP, e depois produção e defesa. A quarta (Go) pode ser trocada por outra especialização, como IA aplicada, mobile avançado, dados ou segurança, sem afetar o resto.

Segunda: Projeto Final I (da ideia ao MVP)

1. Escolha do projeto: problema real, público, critérios e escopo viável para um semestre
2. Pesquisa e validação: concorrentes, entrevistas com possíveis usuários e hipótese de valor
3. Requisitos, histórias de usuário, priorização e definição do MVP

4. Arquitetura: diagramas C4, registros de decisão (ADR) e justificativa da stack
5. Modelagem de dados e contratos de API com OpenAPI
6. Design de interface: wireframes e protótipo no Figma
7. Estrutura do repositório: CI, qualidade de código, ambientes e infraestrutura base
8. Sprint 1: núcleo do back-end (domínio e persistência)
9. Sprint 2: autenticação, segurança e API completa
10. Sprint 3: front-end ou mobile consumindo a API
11. Sprint 4: funcionalidades diferenciais (IA, dados ou tempo real, conforme o projeto)
12. Revisão do MVP: demonstração, feedback e plano de acabamento

Terça: Preparação para Entrevistas Técnicas

1. Mapa do processo seletivo: etapas, tipos de vaga (estágio e júnior) e como se posicionar
2. Padrões de problemas I: arrays, strings, hash, dois ponteiros e janela deslizante
3. Padrões II: pilhas, filas, listas, árvores e recursão
4. Padrões III: grafos, busca e backtracking
5. Padrões IV: programação dinâmica, heaps, intervalos e bits
6. Live coding: pensar em voz alta, testar, analisar complexidade e simular entrevistas
7. Revisão de fundamentos: perguntas clássicas de SO, redes, banco de dados e POO
8. System design para nível júnior: framework de resposta e estudos de caso
9. Entrevista comportamental: método STAR e suas histórias de projeto
10. Currículo, LinkedIn e GitHub: portfólio e leitura por filtros de triagem (ATS)
11. Busca ativa: networking, indicações, testes técnicos em casa e vagas remotas e internacionais
12. Simulados completos de entrevista e noções de negociação de oferta

Quarta: Programação de Sistemas Concorrentes com Go

1. Por que Go: instalação, módulos, tipos e controle de fluxo
2. Funções, structs, métodos e interfaces
3. Coleções, ponteiros, tratamento de erros e pacotes
4. Concorrência I: goroutines e channels
5. Concorrência II: `select`, `sync` (Mutex e WaitGroup) e `context`
6. Padrões de concorrência: worker pool, pipeline, fan-out/fan-in e race detector
7. Servidores HTTP e APIs REST com `net/http`
8. Persistência, testes e benchmarks

9. gRPC e Protocol Buffers
10. Profiling e otimização: `pprof`, coleta de lixo e uso de memória
11. Ferramentas de linha de comando e imagens Docker enxutas
12. Projeto: um serviço de alto desempenho em Go integrado ao projeto final (por exemplo, um worker de filas)

Quinta: Carreira, Comunicação Técnica e Open Source

1. Inglês técnico: ler documentação, escrever issues, commits e pull requests, e um plano de estudo contínuo
2. Escrita técnica: README, relatórios e artigos para blog
3. Open source I: como funciona, licenças e como escolher um projeto
4. Open source II: abrir seu primeiro pull request e passar por code review
5. Trabalho em equipe: Scrum na prática, estimativas e feedback
6. Produtividade e aprendizado contínuo
7. Produto e negócio: como a empresa ganha dinheiro, métricas e valor entregue
8. Ética, direito e profissão: licenças de software, LGPD aplicada e IA responsável
9. Freelancing e empreendedorismo: precificação, contratos e formalização
10. Caminhos de especialização: back-end, dados, mobile, DevOps e IA; certificações e pós-graduação
11. Comunidade: eventos, palestras, mentoria e blog
12. Plano de carreira para os 12 meses seguintes e retrospectiva do curso

Sexta: Projeto Final II (produção e defesa)

1. Estratégia de testes do projeto e meta de cobertura
2. Testes de carga e desempenho, e correção de gargalos
3. Auditoria de segurança com OWASP e correções
4. Deploy em produção: infraestrutura como código, pipeline e ambientes de staging e produção
5. Observabilidade: logs, métricas, alertas e painel de monitoramento
6. Documentação: README, diagramas, OpenAPI e guia de contribuição
7. Relatório técnico no estilo de um TCC: introdução, metodologia, resultados e conclusões
8. Vídeo de demonstração, página de apresentação e estudo de caso para o portfólio
9. Usuários reais: testes de usabilidade e coleta de feedback
10. Segunda rodada de melhorias com base no feedback

11. Apresentação: slides, narrativa e treino do pitch técnico
12. Defesa final diante de uma banca (colegas, professores, mentores ou comunidade) e retrospectiva do curso

Conexões, pontos de atenção e próximos passos

O que se aprende em um semestre vira ferramenta no seguinte, e os projetos de cada um alimentam o projeto final.

Como as matérias se conectam

- A álgebra booleana de Matemática Discreta vira circuito em Arquitetura (Semestre 1).
- O código em C é visto como assembly em Arquitetura, e esse conhecimento volta em escalonamento e memória virtual de Sistemas Operacionais (Semestres 1 e 2).
- Os grafos de Matemática Discreta reaparecem em Algoritmos Avançados e no roteamento de Redes (Semestres 1 e 2).
- O banco e o Java do Semestre 1 viram a API com JPA do Semestre 2 e o Banco de Dados Avançado do Semestre 3.
- A API do Semestre 2 ganha front-end, auditoria de segurança e deploy na nuvem (Semestre 3), um app mobile e pipelines de dados (Semestre 4) e vira a base do projeto final (Semestre 5).

Pontos de atenção

- **Carga:** são cerca de 580 horas. Se ficar insustentável, as matérias mais fáceis de mover são Cloud e DevOps (Semestre 3), Engenharia de Dados (Semestre 4) e Go (Semestre 5).
- **AWS:** exige cartão de crédito mesmo no nível gratuito. Configure alertas de orçamento no primeiro dia e encerre os recursos depois de cada prática.
- **Mobile no Windows:** os testes são em Android (emulador ou celular). Publicar na Google Play tem uma taxa única de cadastro, e testar iOS de verdade exige um iPhone.
- **Versões:** revise ferramentas e versões antes de cada semestre.
- **Diploma:** continue a graduação; este plano é prática e portfólio, não equivalência de diploma.

Próximos passos

- [ ] Instalar um compilador de C e o VS Code
- [ ] Escolher os materiais gratuitos da aula 1 de Estruturas de Dados em C
- [ ] Testar um formato de sessão (leitura, vídeo ou exercício) e decidir qual funciona melhor